



[Blog](#) / Browser-based attacks

6 browser-based attacks every security team should be prepared for



Dan Green
Content & Research

Sep 5, 2025 | 12 min read

Browser-based attacks

Detection & response

Table of contents



What security teams need to know about the browser-based attack techniques that are the leading cause of breaches today.

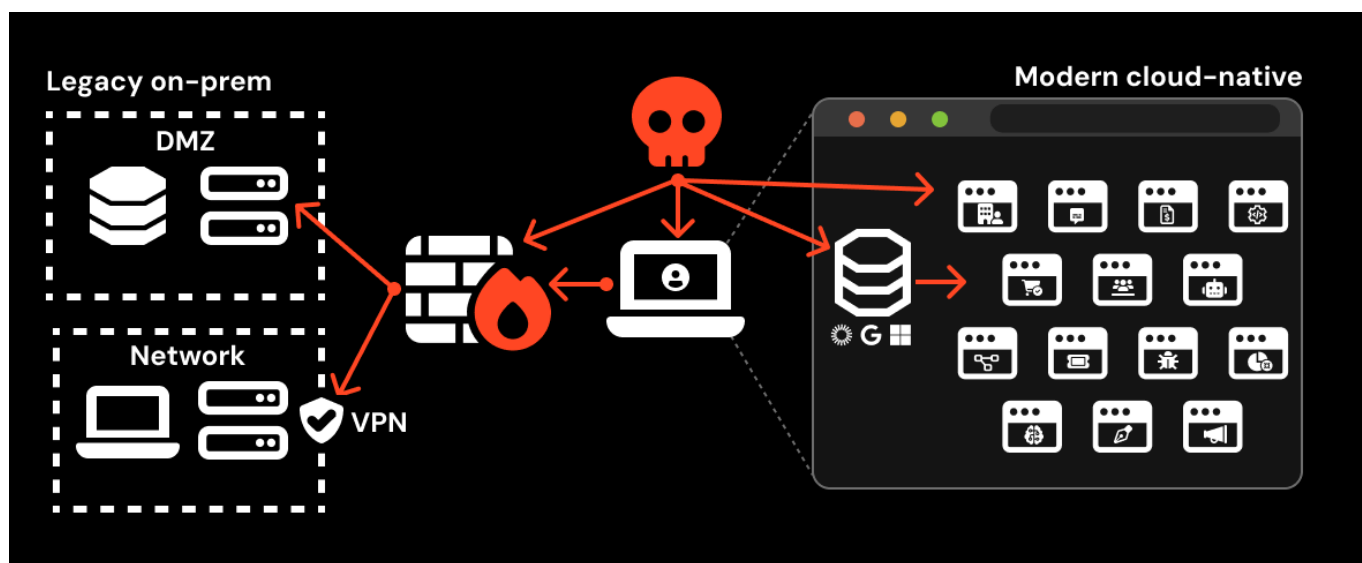
The view that "the browser is the new endpoint" and "the new battleground for cyber attacks" is becoming increasingly advocated by security leaders. But what does this *actually* mean for security teams?

In this article, we're cutting out the jargon to explore what a browser-based attack is, and what's required for effective detection and response.

What is the goal of a browser-based attack?

means going after the third-party apps and services that are now the backbone of business IT — and therefore the top target for attackers.

The most common attack path today sees attackers log into third-party services, dump the data, and monetize it through extortion. You need only look at last year's [Snowflake](#) customer breaches or the still-ongoing Salesforce attacks to see the impact.



Attacks have shifted from targeting local networks to internet services, accessed through employee web browsers.

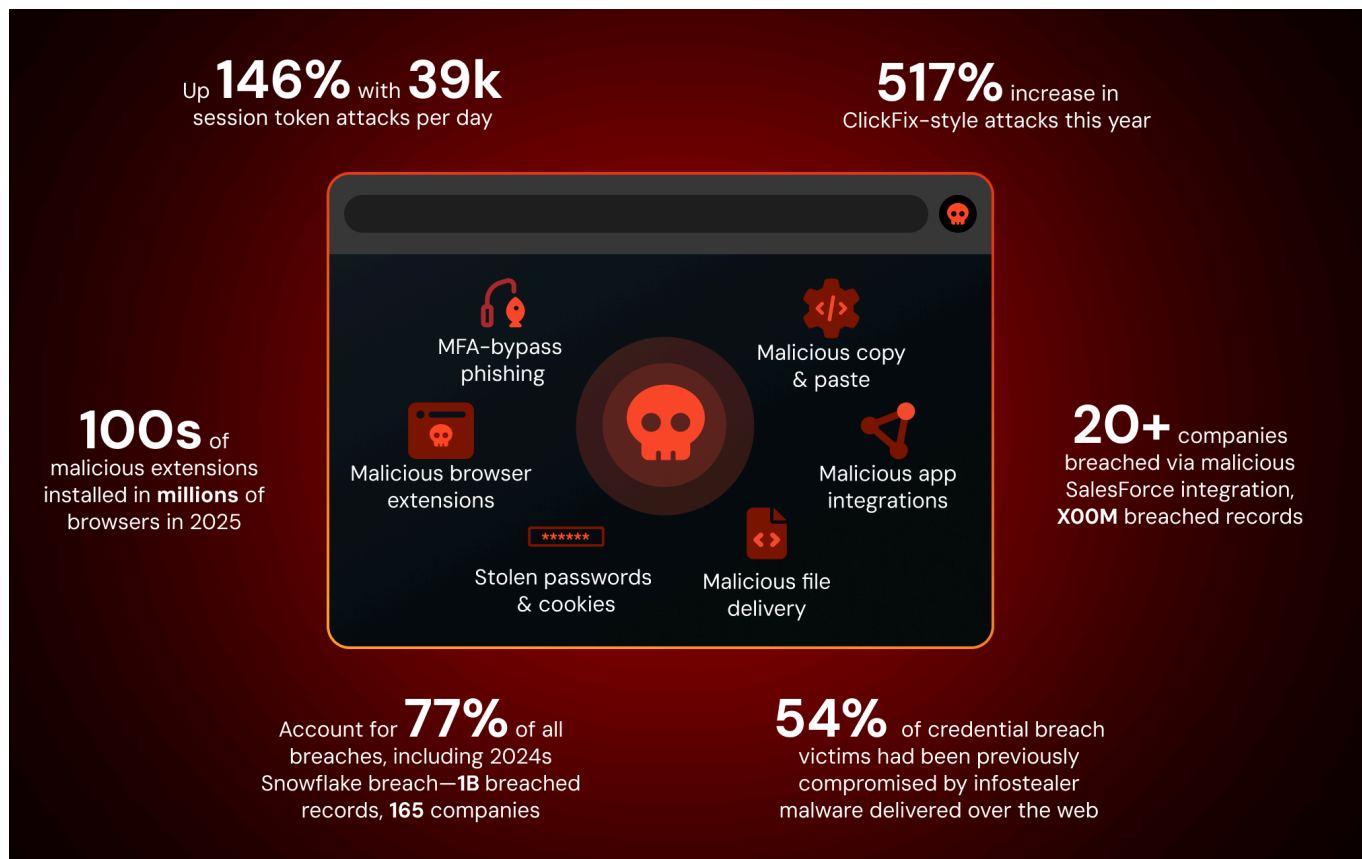
The most logical way to do this is by targeting users of those apps. And because of the changes to working practices, your users are more accessible than ever to external attackers.

Once upon a time, email was the primary communication channel with the wider world, and work happened locally — on your device, and inside your locked-down network environment. This made email and the endpoint the highest priority from a security perspective. But now, with modern work happening across a network of decentralized internet apps, and more varied communication channels outside of email, it's harder to stop users from interacting with malicious content (at least, without significantly impeding their ability to do their jobs).

based attack techniques being used by attackers in the wild today.

The 6 key browser-based attacks that security teams need to know about

Attacks that target users in their web browsers have seen an unprecedented rise in recent years.

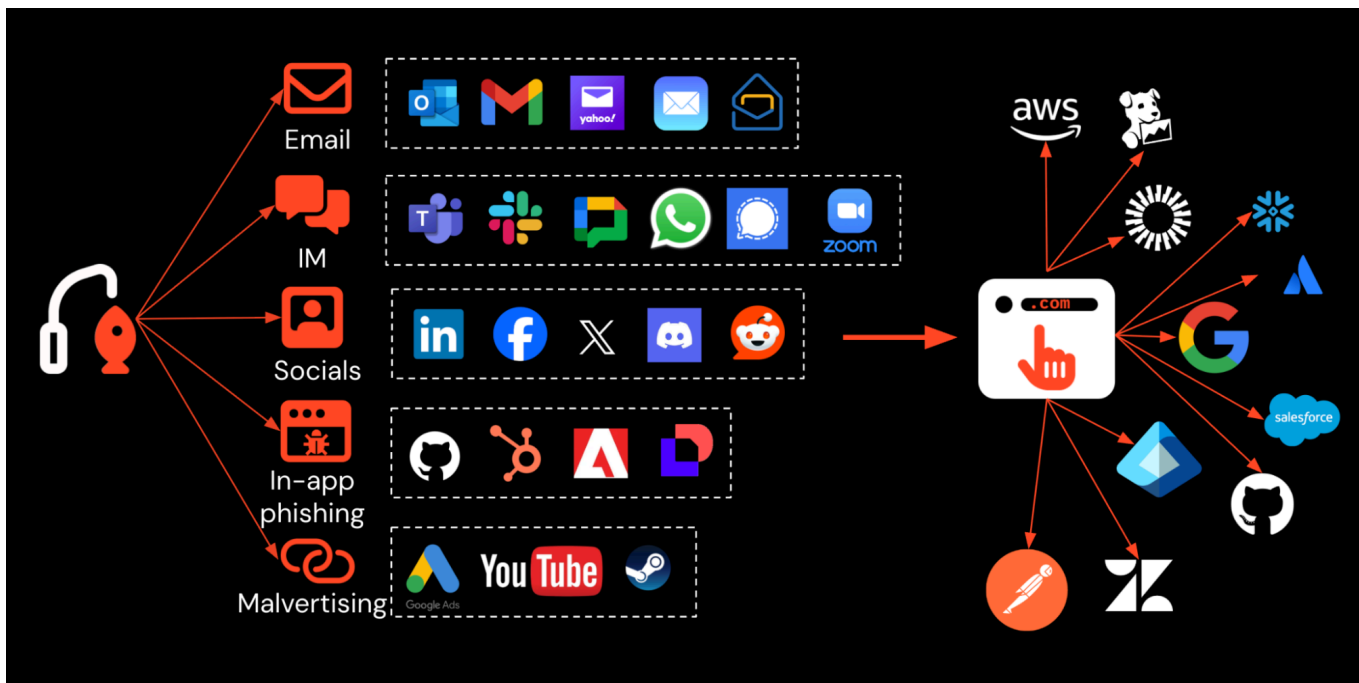


Browser-based attacks like AITM phishing, ClickFix, and consent phishing are the fastest-growing threats of 2025.

Here's our breakdown of the top 6 browser-based attacks that should be on every security team's radar right now.

phish a user of that app. You might not necessarily think of phishing as a browser-based attack, but that's exactly what it is today.

Phishing tooling and infrastructure has evolved a lot in the past decade, while the changes to business IT means there are both many more vectors for phishing attack delivery, and apps and identities to target. Attackers can deliver links over instant messenger apps, social media, SMS, malicious ads, and using in-app messenger functionality, as well as sending emails directly from SaaS services to bypass email-based checks. Likewise, there are now hundreds of apps per enterprise to target, with varying levels of account security configuration.



Phishing is now multi- and cross-channel, targeting a vast range of cloud and SaaS apps using flexible AitM toolkits — but all roads inevitably lead to the browser.

Whereas phishing was once entirely focused on credential theft, modern phishing attacks see the attacker intercept the victim's session on the target app, using reverse-proxy Attacker-in-the-Middle kits that are the standard choice for attackers today. This means most forms of MFA can be bypassed, with the exception of passkeys (though attackers are finding ways to work around passkeys using [downgrade attacks](#)).



AitM kits proxy information to the real site in order to complete the login process, passing MFA checks to steal the user's session.

There are other key differences to be aware of too. Today, phishing operates on an industrial scale, using an array of obfuscation and detection evasion techniques. The latest generation of fully customized AitM phishing kits are dynamically obfuscating the code that loads the web page, implementing custom bot protection (e.g. CAPTCHA or Cloudflare Turnstile), using runtime anti-analysis features, and using legitimate SaaS and cloud services to host and deliver phishing links to cover their tracks.

This means that traditional anti-phishing tools at the email and network layer are struggling to keep up, with many attacks evading email-based detections (or bypassing email altogether). At the same time, proxy-based solutions now see a garbled mess of JavaScript code without the necessary context of what is actually happening in the browser to be able to piece it together effectively. Even if they don't realize it, this means many organizations are now relying solely on blocking known-bad sites and hosts — a wildly ineffective solution in 2025 with the rate that attackers refresh and rotate their phishing infrastructure.

These changes make phishing more effective than ever, and increasingly difficult to detect and block without being able to observe and analyze web pages that a user interacts with in real time — something only possible with browser-level visibility.

attacks have evolved and why they're so effective at evading detection controls.

Get the Whitepaper

2. Malicious copy and paste (aka. ClickFix, FileFix, etc.)

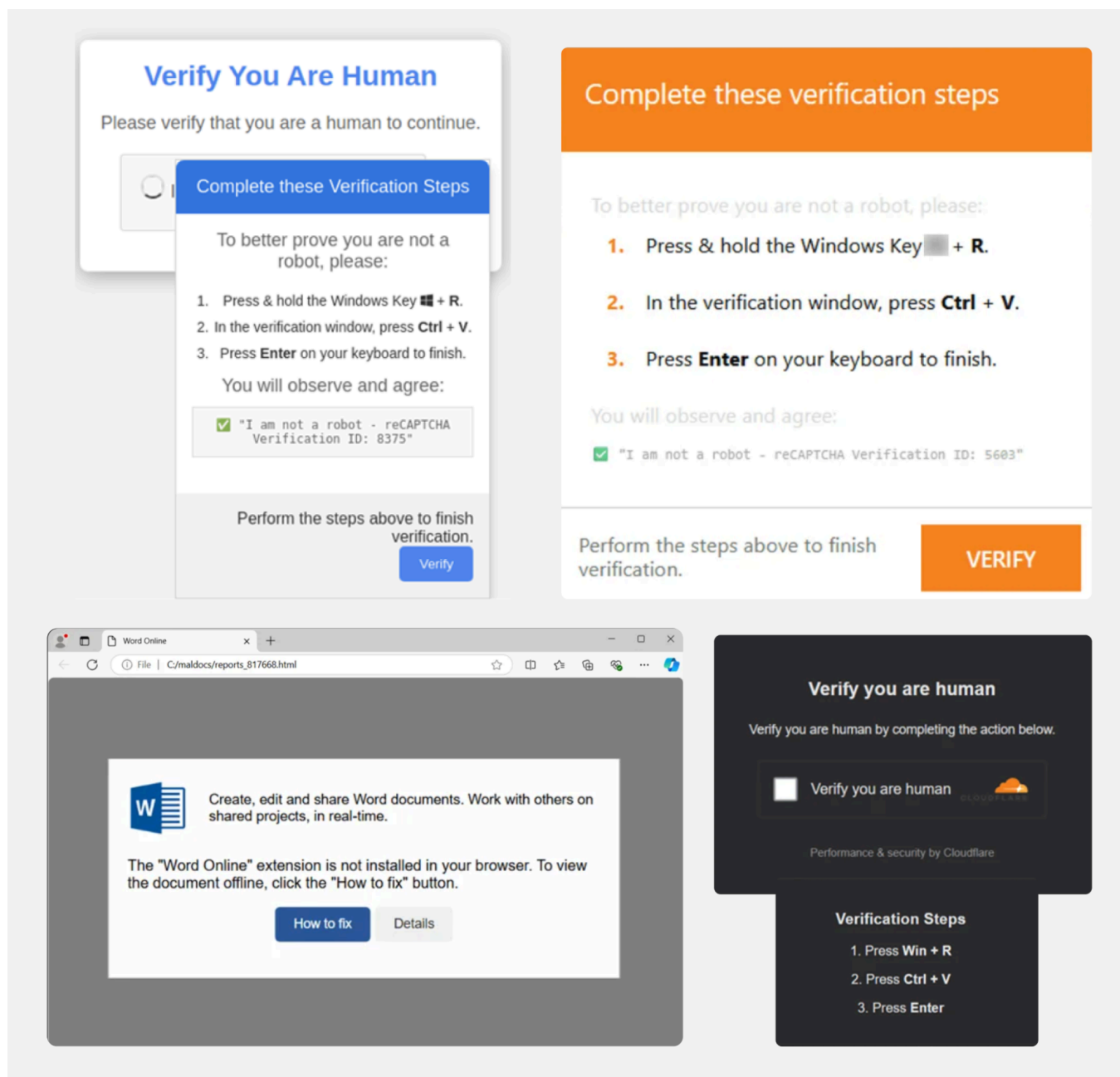
One of the biggest security trends in the past year has been the emergence of the attack technique known as [ClickFix](#).

Originally known as “Fake CAPTCHA”, these attacks attempt to trick users into running malicious commands on their device — typically by solving some form of verification challenge in the browser.

In reality, by solving the challenge, the victim is actually copying malicious code from the page clipboard and running it on their device. It typically gives the victim instructions that involve clicking prompts and copying, pasting, and running commands directly in the Windows Run dialog box, Terminal, or PowerShell. Variants such as [FileFix](#) have also emerged which instead uses the File Explorer Address Bar to execute OS commands, while recent examples have seen this attack branch out to [Mac via the macOS terminal](#).

Most commonly, these attacks are used to deliver infostealer malware, using stolen session cookies and credentials to access business apps and services.

Like modern credential and session phishing, links to malicious pages are distributed over various delivery channels and using a variety of lures, including



Examples of ClickFix lures used by attackers in the wild.

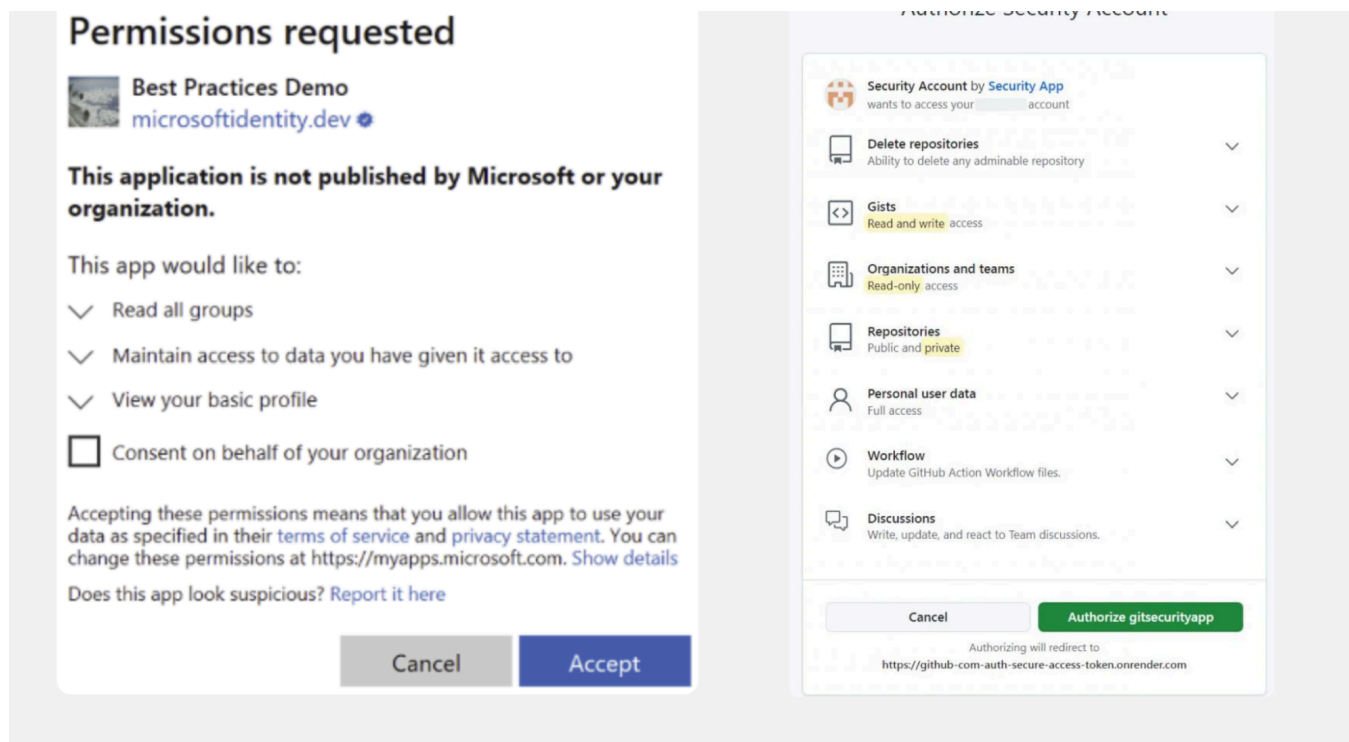
The variance in lure, and differences between different versions of the same lure, can make it difficult to fingerprint and detect based on visual elements alone. Also, many of the same protections being used to obfuscate and prevent analysis of phishing pages also apply to ClickFix pages, making it equally challenging to detect and block them.

are being routinely bypassed, or perhaps evaded altogether by targeting personal or BYOD devices.

There is a significant opportunity to detect these attacks in the browser and stop them at the earliest opportunity, before they reach the endpoint. Every ClickFix attack and variant has a key action in common — malicious code is copied from the page's clipboard. In some cases, this happens without any user interaction (where the only requirement on the user is to run code that has been silently copied behind the scenes), presenting a strong indicator of malicious behavior that can be observed in the browser.

3. Malicious OAuth integrations

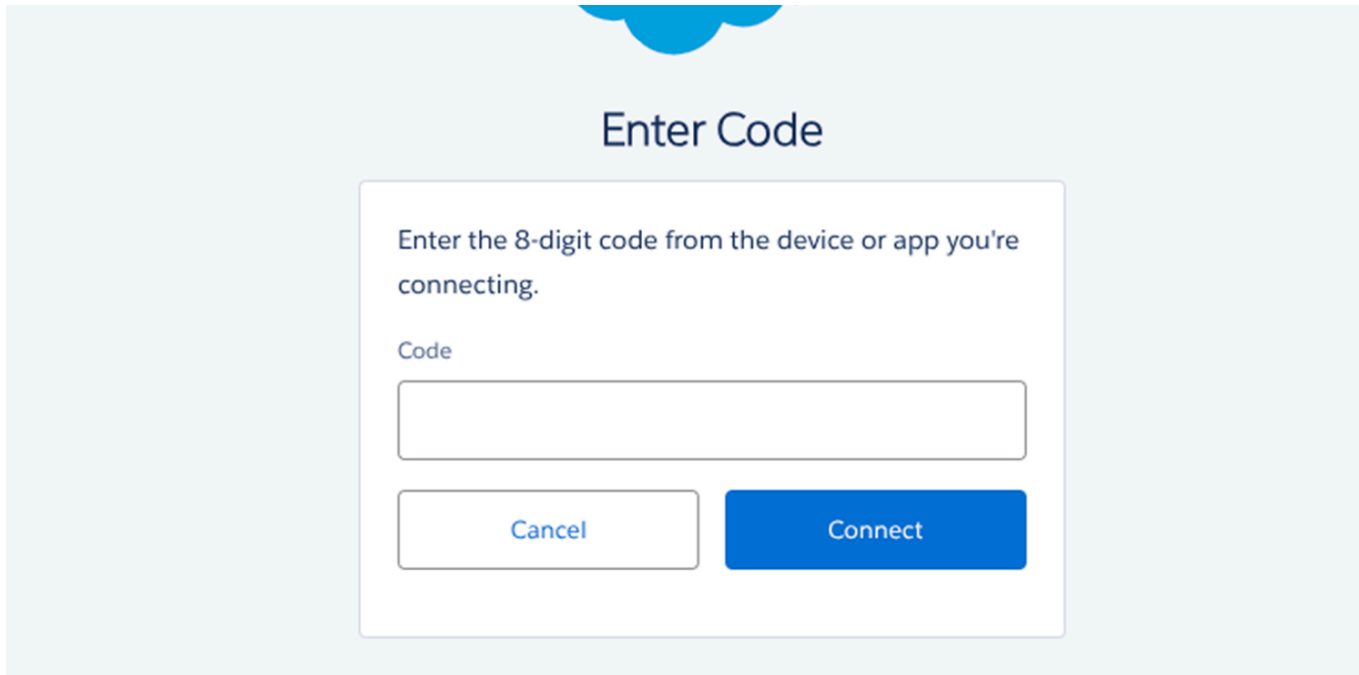
Malicious OAuth integrations are another way for attackers to compromise an app by tricking a user into authorizing an integration with a malicious, attacker-controlled app, with the level of data access and functionality dictated by the scopes authorized in the request. This is also known as [consent phishing](#).



Consent phishing examples, where an attacker tricks the victim into authorizing an attacker-controlled app with risky permissions.

This is an effective way for attackers to bypass hardened authentication and access controls by sidestepping the typical login process to take over an account and compromise business apps. This includes phishing-resistant MFA methods like passkeys — since the standard login process does not apply.

A variant of this attack has dominated the headlines recently with the ongoing Salesforce breaches. In this scenario, the attacker tricked the victim into authorizing an attacker-controlled OAuth app via the device code authorization flow in Salesforce, which requires the user to enter an 8-digit code in place of a password or MFA factor.



The ongoing Salesforce attacks involve malicious OAuth apps being granted access to the victim's Salesforce tenant.

Preventing malicious OAuth grants being authorized requires tight in-app management of user permissions and tenant security settings. This is no mean feat when considering the 100s of apps in use across the modern enterprise, many of which are not centrally managed by IT and security teams (or in some cases, are completely unknown to them). Even then, you're limited by the controls made available by the app vendor. In this case, Salesforce has announced planned changes to OAuth app authorization in order to improve security prompted by these attacks — but many more apps with insecure configs exist for attackers to take advantage of in future.

However, unlike app-specific integrations, browser-based security tools are well positioned to observe OAuth grants across all apps accessed in the browser — even the ones the security team doesn't manage or know about, or without needing to pay for the app's special security add-on to get visibility.

business apps by observing and capturing logins as they happen, and/or extracting session cookies and credentials saved in the browser cache and password manager.

Attackers do this by creating their own malicious extension and tricking your users into installing it, or taking over an existing extension to gain access to browsers where it is already installed ([it's very easy for attackers to buy and add malicious updates to existing extensions](#), easily passing extension web store security checks).

The news around extension-based compromises has been on the rise since the [Cyberhaven extension](#) was hacked in December 2024, along with at least 35 other extensions. Since then, there has been regular reporting on data-stealing extensions [impersonating legitimate brands](#), and [impacting millions of users](#).

Risky browser extension permissions include broad data access, the ability to modify website content, track user activity, capture screenshots, and manage tabs or network requests. Permissions like "read and change all data on all websites" or access to cookies and browsing history are particularly dangerous as they can be exploited for session hijacking, data theft, malware injection, or phishing.

Generally, your employees should not be randomly installing browser extensions unless pre-approved by your security team. The reality, however, is that many organizations have very little visibility of the extensions their employees are using, and the potential risk they're exposed to as a result.

To tackle malicious extensions, security tools operating in the browser can track the browser extensions deployed, highlight risky permissions, compare with known-malicious extensions, identify fraudulent/unofficial versions of a legitimate extension, and highlight other risky properties commonly associated with malicious extensions (e.g. "Developer" extensions).

many years. Just as non-email channels like malvertising and drive-by attacks are used to deliver phishing and ClickFix lures, malicious files are also distributed through similar means — leaving malicious file detection to basic known-bad checks, sandbox analysis using a proxy (not that useful in the context of sandbox-aware malware) or runtime analysis on the endpoint.

This doesn't just have to be malicious executables directly dropping malware onto the device. File downloads can also contain additional links taking the user to malicious content. In fact, one of the most common types of downloadable content are HTML Applications (HTAs), commonly used to spawn local phishing pages to stealthily capture credentials. More recently, attackers have been weaponizing SVG files for a similar purpose, running as self-contained phishing pages that render fake login portals entirely client-side.

Even if malicious content cannot always be flagged from surface-level inspection of a file, recording file downloads in the browser is a useful addition to endpoint-based malware protection, and provides another layer of defense against file downloads that perform client-side attacks, or redirect the user to malicious web-based content.

6. Stolen credentials and MFA gaps

This last one isn't so much a browser-based attack, but it is a product of them. When credentials are stolen through phishing or infostealer malware they can be used to take over accounts missing MFA.

This isn't the most sophisticated attack, but it's very effective. You need only look at last year's [Snowflake](#) account compromises or the [Jira](#) attacks earlier this year to see how attackers harness stolen credentials at scale.

With the modern enterprise using hundreds of apps, the likelihood that an app hasn't been configured for mandatory MFA (if possible) is high. And even when an

(e.g. Google, Microsoft, Okta) and SSO-connected apps doesn't give you a full picture of your identity surface.

Logins can also be observed in the browser — in fact, it's as close to a universal source of truth as you're going to get about how your employees are actually logging in, which apps they're using, and whether MFA is present, enabling security teams to find and fix vulnerable logins before they can be exploited by attackers.

Conclusion

Attacks are increasingly happening in the browser. That makes it the perfect place to detect and respond to these attacks. But right now, the browser is a blind-spot for most security teams.

Push Security's browser-based security platform provides comprehensive detection and response capabilities against the leading cause of breaches. Push blocks browser-based attacks like AiTM phishing, credential stuffing, password spraying and session hijacking using stolen session tokens. You can also use Push to find and fix vulnerabilities across the apps that your employees use, like ghost logins, SSO coverage gaps, MFA gaps, vulnerable passwords, risky OAuth integrations, and more to harden your identity attack surface.

If you want to learn more about how Push helps you to detect and stop attacks in the browser, [book some time with one of our team for a live demo](#).